DropGuard Lite: Log-Based Bot Risk Scoring Using Real Human Checkout Sessions

Godfred Tekpor

February 19, 2026

1 Project Topic Proposal

Limited release clothing drops create a short, high intensity buying window where automated bots can acquire inventory faster than real shoppers. This reduces fairness for customers and increases operational burden for stores, especially when automated traffic overwhelms the purchase flow. Many commercial defenses rely on large scale signals such as device fingerprinting, third party intelligence feeds, and proprietary anti abuse infrastructure. Those systems are difficult to reproduce in a junior independent study. This project narrows the scope to a measurable, reproducible question: how much bot detection signal can be extracted from server side session logs and interaction timing, especially when the legitimate baseline comes from real human sessions instead of scripted simulations.

Prior work suggests that combining web log evidence with behavioral signals improves robustness against advanced automation. Iliou et al. show that pairing web logs with mouse behavioral biometrics can help detect advanced web bots that attempt to evade detection [2]. Challenge based mechanisms can also help, but they can increase friction and can be attacked, so the system design should be evaluated for usability tradeoffs [1]. Research on modeling bot sessions and arrival processes motivates a session level view of automation behavior that can be analyzed from aggregated request traces [3]. Building on these ideas, DropGuard Lite implements an interpretable, log based risk score and evaluates simple mitigation policies at add to cart and checkout.

What I will build

The core software artifact is a minimal drop style web application with product browsing, cart management, and a simulated checkout flow. The application logs each request into an event table keyed by a random session identifier. A feature extraction pipeline aggregates events into per session features that capture speed, navigation path structure, and burst behavior. An interpretable risk score in the range $[0, 1]$ is computed from weighted thresholds over these features. Finally, the application applies mitigation gates at sensitive actions such as add to cart and checkout, using a small set of policies such as allow, delay, or throttle or block.

Inputs and outputs

Inputs include HTTP requests from two sources. The first source is real human sessions collected from volunteers using the site normally during controlled drop windows. The second source is automated bot sessions generated by a fast path checkout script that attempts product to cart to checkout with minimal delay. Outputs include the SQLite event log, a per session feature table, a per session risk score table, and experiment summaries that report bot share of inventory, real human checkout success rate, false positives, and friction measures such as added time or abandonment after mitigation.

Event logging schema

Each request produces one event row with the following fields: event_id, timestamp, session_id, endpoint, method, status_code, latency_ms, and optional user_agent and metadata_json. The goal is to keep the logging schema simple enough to implement correctly, while still supporting meaningful per session analysis.

Real human sessions with permission

A defining change in the revised proposal is that legitimate behavior is collected from real human sessions, not from a scripted human like simulator. Volunteers will see a consent screen before any telemetry is logged. The consent screen explains what is recorded, why it is recorded, and how it is stored. The system will not collect names, emails, addresses, payment information, or any direct identifiers. Checkout is simulated and records purchase success without processing payments, and participants will be instructed not to enter real personal data. Session identifiers are random and are not tied to identity. If the institution requires IRB or departmental review, I will follow the required process before collecting any human data.

Bot variants in scope

To keep the project achievable while still realistic, I will implement one bot type in scope. The fast path checkout bot navigates product to cart to checkout with minimal delay. A stealth pacing bot that adds random delays and more normal navigation is a stretch goal if time permits.

Evaluation plan

Evaluation will use repeated drop runs that mix real human sessions and bot sessions. Each run uses a fixed inventory count and a scheduled drop start time. I will compare mitigation thresholds and policies by measuring bot share of inventory, real human checkout success rate, false positive rate for humans who are delayed or blocked, and friction metrics such as added time to checkout and abandonment after mitigation. The intended outcome is a clear, reproducible tradeoff curve rather than a claim of universal bot prevention.

Figures

Figure 1 shows the end to end system flow from requests to logging, feature extraction, risk scoring, mitigation, and metrics. Figure 2 shows a UI sketch for the drop site, including the consent notice and the basic purchase flow.

Figure 1: Proposed architecture for DropGuard Lite. The system logs session events, extracts per session features, computes a risk score, and applies mitigation at add to cart and checkout.

2 Appendix: Feature List (Development Roadmap)

Planned features for DropGuard Lite, in completion order:

1. Implement product listing and product detail endpoints with fixed inventory and scheduled drop start time.
2. Implement cart endpoints (add, remove, update quantity) tied to a session ID cookie.
3. Implement simulated checkout that records purchase success and decrements inventory without collecting payment data.
4. Implement an event logging layer that stores one row per request in SQLite using the defined schema.
5. Implement a consent screen that requires explicit permission before any session telemetry is logged.

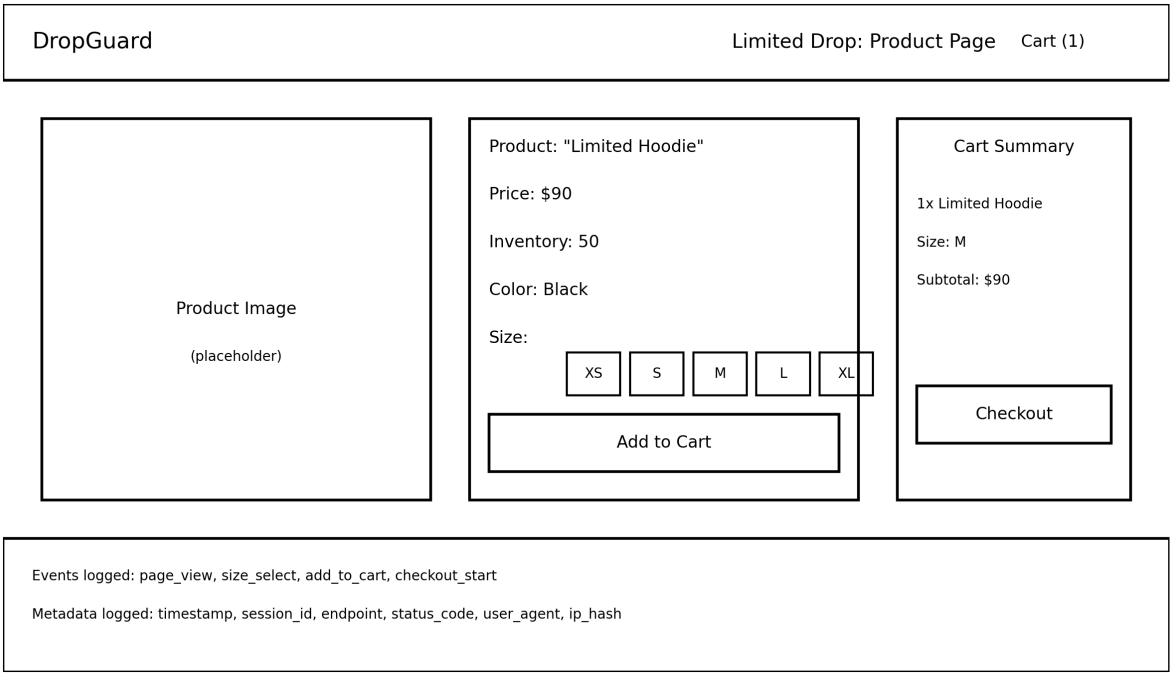

Figure 2: Prototype UI sketch for the drop site. The interface supports product viewing, cart actions, a simulated checkout flow, and a consent notice for session telemetry logging.

6. Implement a data export script that outputs a clean session event table for analysis.
7. Implement per session feature extraction (speed features, path features, burst features).
8. Implement an interpretable risk scoring function that maps features to a score in $[0, 1]$.
9. Implement mitigation gates at add to cart and checkout (allow, delay, throttle or block) with configurable thresholds.
10. Implement a fast checkout bot client script to generate automated traffic.
11. Implement an experiment runner that mixes bot scripts with real human sessions and runs repeated trials.
12. Implement evaluation metrics and produce summary tables and plots (bot inventory share, human success rate, false positives, friction).

Stretch goals if time permits:

1. Add a stealth pacing bot variant that randomizes timing and navigation to stress test the risk score.
2. Add a read only monitoring page showing recent sessions, features, and risk scores.
3. Replace fixed delays with a simple queue mechanism for high risk sessions.

References

[1] Gangwal, A., Reddy, P. S., and Sagar, C. y. k. *Swiss Cheese CAPTCHA: A Novel Multi-barrier Mechanism for Bot Detection.* Association for Computing Machinery, New York, NY, USA, 2025, pp. 1780–1789.

[2] Iliou, C., Kostoulas, T., Tsikrika, T., Katos, V., Vrochidis, S., and Kompatsiaris, I. Detection of advanced web bots by combining web logs with mouse behavioural biometrics. *Digital Threats 2*, 3 (June 2021).

[3] Iliou, C., Kostoulas, T., Tsikrika, T., Katos, V., Vrochidis, S., and Kompatsiaris, I. Web bot detection evasion using deep reinforcement learning. In *Proceedings of the 17th International Conference on Availability, Reliability and Security* (New York, NY, USA, 2022), ARES '22, Association for Computing Machinery.